SAECOIN

GITHUB REPOSITORY STRUCTURE

Version 1.0

Project: SAECOIN
Ticker: SAC (Provisional)
Repository Name: saecoin
Repository Type: Monorepo
Primary Language: Rust
Mobile Application: Kotlin + Jetpack Compose
Desktop Application: Tauri + TypeScript
Explorer Frontend: React + TypeScript
Status: Planning

---

1. PURPOSE

This document defines the official proposed GitHub repository structure for SAECOIN.

The repository will use a monorepo architecture, meaning the main blockchain software and related applications will be maintained in one GitHub repository.

The structure must support:

- Blockchain core.
- Full node.
- Consensus.
- UTXO transaction system.
- P2P networking.
- CPU mining.
- GPU mining.
- Wallet core.
- Android wallet.
- Windows desktop wallet.
- API services.
- Block explorer.
- Automated tests.
- Documentation.
- Build and release tools.

The structure is designed to support future expansion without requiring an unnecessary repository migration.

2. REPOSITORY OVERVIEW

Proposed GitHub repository:

"saecoin"

Proposed top-level structure:

```
saecoin/
│
├── .github/
├── docs/
├── crates/
├── apps/
├── services/
├── tests/
├── scripts/
├── configs/
├── assets/
├── tools/
├── Cargo.toml
├── Cargo.lock
├── rust-toolchain.toml
├── .gitignore
├── .editorconfig
├── README.md
├── LICENSE
└── CONTRIBUTING.md
```

This structure uses a Rust workspace for shared blockchain components.

3. DOCUMENTATION DIRECTORY

```
docs/
│
├── product/
│   ├── PRD.md
│   ├── AppFlow.md
│   └── UI_Spec.md
│
├── technical/
│   ├── TRD.md
│   ├── Schema.md
│   ├── Blockchain_Architecture.md
│   ├── GitHub_Repository_Structure.md
│   └── Coding_Standards.md
│
├── consensus/
│   ├── Consensus_Spec.md
│   ├── Network_Parameters.md
```

```
│   ├── Genesis_Block_Spec.md
│   └── Test_Vectors.md
│
├── security/
│   ├── Security_Spec.md
│   ├── Wallet_Security_Design.md
│   └── Security_Audit.md
│
├── api/
│   └── API_Spec.md
│
├── testing/
│   └── Testing_Strategy.md
│
├── development/
│   ├── Development_Roadmap.md
│   ├── Build_and_Release.md
│   └── Development_Setup.md
│
└── operations/
    ├── Node_Operations.md
    ├── Mining_Guide.md
    └── Mainnet_Launch_Checklist.md
```

All project specifications should be maintained in this directory.

4. RUST WORKSPACE

The "crates/" directory will contain reusable Rust libraries.

Each crate should have a clearly defined responsibility.

```
crates/
│
├── saecoin-primitives/
├── saecoin-crypto/
├── saecoin-consensus/
├── saecoin-transaction/
├── saecoin-utxo/
├── saecoin-blockchain/
├── saecoin-storage/
├── saecoin-mempool/
├── saecoin-p2p/
├── saecoin-rpc/
├── saecoin-wallet-core/
├── saecoin-mining-core/
├── saecoin-miner-cpu/
├── saecoin-miner-gpu/
```

```
├── saecoin-config/
└── saecoin-test-utils/
```

4.1 saecoin-primitives

Contains fundamental data types.

Responsibilities:

- Amount types.
- Hash types.
- Block identifiers.
- Transaction identifiers.
- Network identifiers.
- Common protocol types.

4.2 saecoin-crypto

Contains cryptographic interfaces and implementations.

Responsibilities:

- Hashing.
- Digital signatures.
- Key-related cryptographic operations.
- Secure randomness interfaces.

Cryptographic algorithms must follow the finalized protocol specification.

4.3 saecoin-consensus

Contains consensus validation.

Responsibilities:

- Proof-of-work verification.
- Block validation rules.
- Difficulty adjustment.
- Block subsidy.
- Halving calculation.
- Timestamp validation.
- Chain selection rules.

This crate must not depend on wallet UI or explorer functionality.

4.4 saecoin-transaction

Contains transaction structures and processing rules.

Responsibilities:

- Transaction serialization.
- Transaction identifiers.
- Inputs.
- Outputs.
- Transaction validation.
- Fee calculation.

4.5 saecoin-utxo

Contains UTXO state management.

Responsibilities:

- UTXO creation.
- UTXO spending.
- UTXO lookup.
- Balance-related primitives.
- Double-spend prevention.

4.6 saecoin-blockchain

Contains blockchain management.

Responsibilities:

- Block management.
- Chain state.
- Active chain.
- Block connection.
- Block disconnection.
- Reorganization coordination.

4.7 saecoin-storage

Contains database access.

Responsibilities:

- Block storage.
- Transaction storage.
- UTXO storage.
- Block indexes.
- Undo data.
- Database recovery.

The database engine remains configurable until benchmarking confirms the final choice.

4.8 saecoin-mempool

Contains unconfirmed transaction management.

Responsibilities:

- Transaction admission.
- Fee policy.
- Conflict detection.
- Transaction expiration.
- Mempool limits.

4.9 saecoin-p2p

Contains peer-to-peer networking.

Responsibilities:

- Peer connections.
- Handshake.
- Message framing.
- Peer discovery.
- Block propagation.
- Transaction propagation.
- Synchronization.

4.10 saecoin-rpc

Contains shared RPC and API types.

Responsibilities:

- Request structures.
- Response structures.
- Error structures.
- API serialization.
- Protocol versioning.

4.11 saecoin-wallet-core

Contains reusable wallet logic.

Responsibilities:

- Key management.
- Address generation.

- Transaction construction.
- Local signing.
- Wallet backup interfaces.
- Wallet recovery interfaces.

4.12 saecoin-mining-core

Contains shared mining logic.

Responsibilities:

- Mining jobs.
- Nonce handling.
- Target handling.
- Solution validation interfaces.
- Mining statistics.

4.13 saecoin-miner-cpu

Contains CPU mining implementation.

4.14 saecoin-miner-gpu

Contains GPU mining implementation.

GPU-specific dependencies must remain isolated from the main blockchain consensus implementation.

4.15 saecoin-config

Contains shared configuration structures and validation.

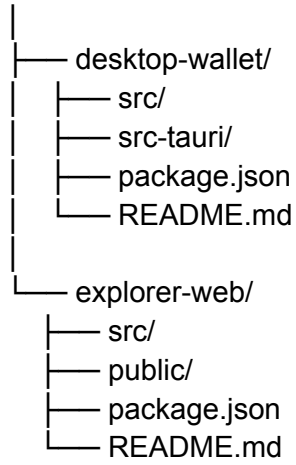
4.16 saecoin-test-utils

Contains shared testing utilities and fixtures.

5. APPLICATION DIRECTORY

The "apps/" directory contains end-user applications.

```
apps/
│
├── android-wallet/
│   ├── app/
│   ├── gradle/
│   ├── build.gradle.kts
│   ├── settings.gradle.kts
│   └── README.md
```



5.1 Android Wallet

Technology:

- Kotlin.
- Jetpack Compose.
- Android SDK.
- Rust wallet core integration where practical.

The Android wallet must not implement a separate, inconsistent transaction validation protocol.

5.2 Windows Desktop Wallet

Technology:

- Tauri.
- Rust.
- TypeScript.

The desktop wallet will reuse shared Rust wallet logic wherever practical.

5.3 Explorer Web

Technology:

- React.
- TypeScript.
- Modern web tooling.

The explorer frontend will communicate with the explorer backend through documented APIs.

6. SERVICES DIRECTORY

The "services/" directory contains independently runnable services.

```
services/
│
├── saecoin-node/
│   ├── src/
│   └── Cargo.toml
│
├── saecoin-miner/
│   ├── src/
│   └── Cargo.toml
│
├── saecoin-explorer/
│   ├── src/
│   └── Cargo.toml
│
└── saecoin-api/
    ├── src/
    └── Cargo.toml
```

6.1 saecoin-node

Main full-node executable.

Responsibilities:

- Initialize blockchain.
- Load configuration.
- Start database.
- Validate blocks.
- Manage mempool.
- Connect to P2P network.
- Synchronize blockchain.
- Provide local RPC access.

6.2 saecoin-miner

Mining application entry point.

Responsibilities:

- Configure CPU mining.
- Configure GPU mining.
- Connect to a node.
- Receive mining jobs.
- Submit solutions.
- Display mining statistics.

6.3 saecoin-explorer

Explorer backend.

Responsibilities:

- Read blockchain data.
- Build indexes.
- Provide block information.
- Provide transaction information.
- Provide address information.
- Provide network statistics.

6.4 saecoin-api

API service.

Responsibilities:

- HTTP API.
- API versioning.
- Authentication where required.
- Rate limiting.
- Request validation.
- Response formatting.

The node's consensus validation must remain authoritative. API services must not independently create or approve consensus rules.

7. TESTS DIRECTORY

tests/
│
├── consensus/
├── transactions/
├── utxo/
├── blockchain/
├── storage/
├── p2p/
├── mining/
├── wallet/
├── api/
├── explorer/
├── compatibility/
├── fuzz/
└── fixtures/

Testing categories:

- Unit tests.
- Integration tests.
- End-to-end tests.
- Consensus test vectors.
- Regression tests.
- Fuzz tests.
- Compatibility tests.
- Performance tests.

All consensus-critical tests must be reproducible.

8. SCRIPTS DIRECTORY

```
scripts/
│
├── setup/
├── build/
├── test/
├── devnet/
├── testnet/
├── release/
├── database/
└── utilities/
```

Purpose:

- Development setup.
- Build automation.
- Test automation.
- Local network startup.
- Testnet management.
- Release packaging.
- Database maintenance.

Scripts must not contain hardcoded production secrets.

9. CONFIGS DIRECTORY

```
configs/
│
├── development/
├── testnet/
├── mainnet/
└── examples/
```

Configuration files must separate network-specific settings.

Mainnet and testnet must have different:

- Network identifiers.
- Genesis blocks.
- Address prefixes.
- Peer configurations.
- Data directories.
- Network ports, where applicable.

Private keys and secrets must never be committed.

10. GITHUB WORKFLOWS

```
.github/
│
├── workflows/
│   ├── rust-ci.yml
│   ├── android-ci.yml
│   ├── desktop-ci.yml
│   ├── explorer-ci.yml
│   ├── security-audit.yml
│   └── release.yml
│
├── ISSUE_TEMPLATE/
└── PULL_REQUEST_TEMPLATE.md
```

Workflows should eventually perform:

- Formatting checks.
- Compilation.
- Unit tests.
- Integration tests.
- Dependency audits.
- Security checks.
- Application builds.

The workflows should be introduced gradually as the relevant modules become available.

11. ROOT CARGO WORKSPACE

The root "Cargo.toml" will define the Rust workspace.

Illustrative structure:

```
[workspace]
resolver = "2"

members = [
```

```
    "crates/saecoin-primitives",
    "crates/saecoin-crypto",
    "crates/saecoin-consensus",
    "crates/saecoin-transaction",
    "crates/saecoin-utxo",
    "crates/saecoin-blockchain",
    "crates/saecoin-storage",
    "crates/saecoin-mempool",
    "crates/saecoin-p2p",
    "crates/saecoin-rpc",
    "crates/saecoin-wallet-core",
    "crates/saecoin-mining-core",
    "crates/saecoin-miner-cpu",
    "crates/saecoin-miner-gpu",
    "crates/saecoin-config",
    "crates/saecoin-test-utils",
    "services/saecoin-node",
    "services/saecoin-miner",
    "services/saecoin-explorer",
    "services/saecoin-api"
]
```

This is an initial example. The final workspace membership must be verified when the project is initialized.

12. DEPENDENCY RULES

To avoid circular dependencies, use these general rules:

1. "saecoin-primitives" should remain foundational.
2. "saecoin-crypto" should provide cryptographic functionality.
3. "saecoin-transaction" may depend on primitives and crypto.
4. "saecoin-utxo" may depend on primitives and transaction.
5. "saecoin-consensus" may depend on protocol-critical crates.
6. "saecoin-blockchain" may coordinate consensus and storage.
7. "saecoin-p2p" may communicate with node-facing interfaces.
8. Applications should use shared libraries instead of duplicating core logic.
9. Explorer functionality must not modify consensus state.
10. UI code must never become the authority for transaction validity.

The exact dependency graph must be reviewed during architecture implementation.

13. NAMING CONVENTIONS

Rust crate names:

"saecoin-<module>"

Rust source files:

"snake_case.rs"

Rust types:

"PascalCase"

Rust functions:

"snake_case"

Documentation:

"PascalCase_With_Underscores.md"

Application packages and modules must follow their platform conventions.

14. GIT IGNORE POLICY

The ".gitignore" file must exclude:

- Build directories.
- Temporary files.
- Local databases.
- Wallet files.
- Private keys.
- Seed backups.
- Local environment files.
- API secrets.
- IDE temporary files.
- Generated binaries.

Example:

```
/target/
/build/
/dist/
/node_modules/
/.gradle/
/.idea/
/.vscode/

.env
.env.*
!.env.example

*.db
```

*.sqlite
*.wallet
*.seed
*.key
*.pem

.DS_Store
Thumbs.db

Additional patterns must be added as the project grows.

Never rely on ".gitignore" alone to protect secrets. Secret scanning and careful repository review are also required.

15. README REQUIREMENTS

The root "README.md" should include:

1. Project introduction.
2. Project status.
3. Technical overview.
4. Features.
5. Repository structure.
6. Development requirements.
7. Build instructions.
8. Test instructions.
9. Documentation links.
10. Security reporting instructions.

The README must clearly state whether the software is experimental, testnet-only, or production-ready.

16. REPOSITORY SECURITY POLICY

The repository must follow these rules:

- Never commit private keys.
- Never commit seed phrases.
- Never commit passwords.
- Never commit production API secrets.
- Review dependencies before adding them.
- Require review for consensus-critical changes.
- Protect the main branch.
- Use signed release artifacts where practical.
- Publish security advisories responsibly.
- Keep development and production configurations separate.

17. FUTURE EXPANSION

The structure should allow future additions such as:

- Mining pool services.
- Hardware wallet support.
- Additional wallet platforms.
- Mobile light clients.
- Developer SDKs.
- Additional explorer services.
- Network monitoring tools.
- Merchant payment integrations.

These features must not be added before the core blockchain is stable.

18. FINAL REPOSITORY POLICY

The repository must remain modular, understandable, testable, and secure.

The main priority is to keep consensus-critical code isolated from applications, interfaces, and optional services.

The repository structure is a proposed baseline. It may be refined during implementation, but changes must be documented and reviewed before they affect consensus-critical modules.

END OF DOCUMENT